Module 2 - Robot Kinematics: Formula Summary

Curriculum: Physical AI | Lessons: 6-10 | Status: Module completed after targeted reinforcement

Symbols and conventions

• L1, L2: link lengths (meters). theta1: shoulder angle; theta2: elbow angle relative to Link 1. Angles are radians inside Python trigonometric functions.

• (x, y): end-effector position measured from the robot base in the planar XY frame.

• q = [theta1, theta2]^T: joint vector; q_dot: joint velocities (rad/s); p_dot: Cartesian velocity (m/s).

Lesson 6 - Links, joints, degrees of freedom

• A link is a rigid robot body; a joint permits constrained relative motion between connected bodies.

• Revolute joint: angular coordinate (rad or degrees); prismatic joint: linear coordinate (m).

• Joint-space degrees of freedom (DOF): number of independent coordinates needed to specify a robot's configuration. A typical open-chain arm with n independent, single-DOF joints has n joint DOFs.

Lesson 7 - Forward kinematics (FK)

Planar two-link position, with joint 2 measured relative to link 1:

x = L1*cos(theta1) + L2*cos(theta1 + theta2)

y = L1*sin(theta1) + L2*sin(theta1 + theta2)

End-effector orientation: phi = theta1 + theta2 (for a simple planar two-link arm).

Worked case: L1=3 m, L2=2 m, theta1=0 deg, theta2=90 deg -> (x,y)=(3,2) m.

Lesson 8 - Analytical inverse kinematics (IK)

For a desired target (x,y):

r = sqrt(x^2 + y^2)

Geometric reachability: abs(L1-L2) <= r <= L1+L2 (ignores joint limits and obstacles).

c2 = (x^2 + y^2 - L1^2 - L2^2)/(2*L1*L2)

theta2 = +acos(c2) OR theta2 = -acos(c2)

alpha = atan2(y, x)

beta = atan2(L2*sin(theta2), L1 + L2*cos(theta2))

theta1 = alpha - beta

Numerical safeguard: after checking reachability, clamp c2 to [-1, 1] to address tiny floating-point roundoff at boundaries.

Example: L1=4 m, L2=3 m, target (2.5,4.330127...) m gives approximately (23.13 deg,+90 deg) and (96.87 deg,-90 deg); both are valid position solutions under the simplified geometric model.

Lesson 9 - Jacobian and singularities

p_dot = J(q) * q_dot

For the two-link planar arm:

J11 = -L1*sin(theta1) - L2*sin(theta1+theta2)

J12 = -L2*sin(theta1+theta2)

J21 = L1*cos(theta1) + L2*cos(theta1+theta2)

J22 = L2*cos(theta1+theta2)

J = [[J11,J12],[J21,J22]]

det(J) = L1*L2*sin(theta2)

If det(J)=0 for this 2x2 Jacobian, its rank is below 2 and arbitrary instantaneous XY velocities cannot be commanded. The joints are not mechanically locked.

Example: L1=3, L2=2, theta1=0, theta2=90 deg: J=[[-2,-2],[3,0]]. For q_dot=[0.1,0.2] rad/s, p_dot=[-0.6,0.3] m/s.

Lesson 10 - Numerical IK

e_k = p_target - FK(q_k)

delta_q = eta * pinv(J(q_k)) * e_k

q_(k+1) = q_k + delta_q

Repeat until norm(e_k) < tolerance or max_steps reached. pinv is the Moore-Penrose pseudoinverse, and eta is a dimensionless step size.

Cautions: The Jacobian linearization is local; convergence depends on initialization and step size. Near singularities, pseudoinverse updates may be large. Practical solvers may use damping, step limits, joint limits, and collision checks.

Cross-lesson workflow

1. Specify link geometry and joint conventions.
2. Compute FK to locate the current end effector.
3. Check target reachability, then solve analytical or numerical IK.

4. Recompute FK and check position error: norm(FK(q)-target) < tolerance.

5. Use the Jacobian for differential motion and check singular or near-singular configurations.

Python snippets

```
import math
import numpy as np
# Forward kinematics
x = L1*math.cos(t1) + L2*math.cos(t1+t2)
y = L1*math.sin(t1) + L2*math.sin(t1+t2)
# IK position validation
error = math.hypot(x-x_target, y-y_target)
passed = error < 1e-6
# Differential kinematics
p_dot = J @ q_dot
# Numerical IK increment
delta_q = eta * (np.linalg.pinv(J) @ (target - fk(q)))
```

Module 2 lab evidence (reported)

• Lesson 7: Isaac Sim visual two-link scene, with cubes as visual primitives rather than physics-driven joints.

• Lesson 8: both elbow branches verified with FK; unreachable targets rejected; external Python USD controller generated Isaac Sim views for targets (2,3) and (3,2).

• Lesson 9: Jacobian/velocity and rank/singularity Python tests passed.

• Lesson 10: numerical IK convergence and plotted trajectory reported as passed.

Scope note: These examples establish kinematics and visual scene manipulation, not torque control, collision-free planning, or physics-based articulation.